Carrera: Ing. en Computación

PROTOCOLOS DE COMUNICACIONES INDUSTRIALES

2º Cuatrimestre.

Docentes: Mg. Ing. Martin Pico, Ing. Milton Pozzo

Tarea práctica 2 - OPC

COMPETENCIA ESPECÍFICA:

2. Planificar, dirigir, realizar y/o evaluar proyectos de relevamiento, análisis, especificación, diseño, desarrollo, implementación, verificación, validación, puesta a punto, mantenimiento y actualización, para todo tipo de personas físicas o jurídicas, de:

b. Sistemas de comunicación de datos y redes de información

RESULTADO DE APRENDIZAJE:

Que el estudiante sea capaz de:

- Estudiar y conocer cada bloque funcional, sus componentes principales, funciones dentro del sistema.
- Explicar el principio de funcionamiento de las comunicaciones cableadas e inalámbricas y sus tipos de conexionado.

Objetivo: *implementar una comunicación cliente-servidor OPC UA sobre microcontroladores o computadoras embebidas, analizando en detalle la estructura de las tramas del protocolo, sus mecanismos de seguridad y los distintos transportes y medios físicos sobre los que puede operar.*

Objetivos específicos:

Que los estudiantes puedan:

- Diferenciar los elementos que intervienen en una comunicación OPC UA: servidor, cliente, endpoint, canal seguro, sesión, espacio de direcciones y nodos.
- Diseñar un modelo de información propio (objetos, variables, tipos de datos y NodeIds fijos) y publicarlo en un servidor OPC UA.
- Capturar, decodificar e interpretar campo por campo las tramas del transporte binario OPC UA (UA TCP) con un analizador de protocolos.
- Comparar los mecanismos de acceso a datos por lectura cíclica (Read) y por suscripción (Subscription / MonitoredItems).
- • Configurar y evaluar los distintos modos y políticas de seguridad del protocolo.
- • Identificar los transportes, codificaciones y medios físicos admitidos por OPC UA y contrastarlos con la capa física RS485 empleada en la Tarea Nº1 (MODBus RTU).

Marco teórico de referencia

OPC UA (IEC 62541) no define una capa física propia: es una arquitectura orientada a servicios que se apoya en pilas IP/Ethernet existentes. El informe deberá desarrollar, con bibliografía, los conceptos que se resumen a continuación; este apartado es una guía de estudio, no una respuesta.

Modelo en capas de OPC UA. El protocolo separa el modelo de información (espacio de direcciones, nodos, referencias y atributos), los servicios abstractos (Read, Write, Browse, CreateSubscription, Publish, etc.) y los mapeos de transporte, que definen cómo esos servicios se codifican y viajan por la red. Un mismo servicio puede viajar con distintas codificaciones y transportes sin cambiar el modelo de información.

Estructura general de un mensaje UA TCP binario. Toda trama comienza con una cabecera fija de 8 bytes, codificada en little-endian:

Campo	Tamaño	Contenido
MessageType	3 bytes (ASCII)	HEL, ACK, ERR, RHE, OPN, MSG o CLO
ChunkType	1 byte (ASCII)	F (final), C (intermedio) o A (abortado)
MessageSize	4 bytes (UInt32)	Longitud total del mensaje en bytes, cabecera incluida

A partir de OPN, MSG y CLO siguen: el identificador del canal seguro (SecureChannelId), la cabecera de seguridad (asimétrica en OPN, simétrica con TokenId en MSG y CLO), la cabecera de secuencia (SequenceNumber y RequestId) y el cuerpo, que inicia con el NodeId codificado del tipo de servicio, un RequestHeader o ResponseHeader y los parámetros del servicio. Cuando el canal usa Sign o SignAndEncrypt se agregan relleno (padding) y firma al final.

Transportes y medios físicos. La siguiente tabla resume los mapeos que deberán investigarse y profundizarse:

Mapeo	Esquema URL / puerto típico	Codificación	Modelo de comunicación
UA TCP (UA Binary)	opc.tcp:// — 4840	Binaria	Cliente-servidor
HTTPS	https:// — 443	Binaria o JSON	Cliente-servidor
WebSockets seguros	opc.wss:// — 443	Binaria o JSON	Cliente-servidor

PubSub sobre UDP	opc.udp:// — 4840 (multicast)	UADP (binaria)	Publicador-suscriptor sin broker
PubSub sobre Ethernet capa 2	opc.eth:// (EtherType 0xB62C)	UADP	Publicador-suscriptor determinista (con TSN)

Por debajo de IP, OPC UA puede operar sobre cualquier medio que transporte tramas Ethernet o IP: Ethernet de cobre (100BASE-TX, 1000BASE-T), fibra óptica, Wi-Fi (IEEE 802.11), redes celulares 5G, Ethernet de par simple (SPE: 10BASE-T1L / Ethernet-APL para campo y zonas con riesgo de explosión, 10BASE-T1S para buses multipunto cortos) y Ethernet con TSN (IEEE 802.1) para tiempo real determinista. Los mapeos PubSub sobre brokers (MQTT, AMQP) existen en la especificación, pero quedan fuera del alcance de esta tarea.

Consignas

PARTE 1: Montaje de la red Ethernet punto a punto

1. **Conexión física:** conectar ambas Raspberry Pi directamente con un cable Ethernet, sin switch ni router.
2. **Direccionamiento estático:** como no hay servidor DHCP en el enlace, asignar a cada interfaz `eth0` una dirección IP fija:

Equipo	Rol	Dirección IP	Máscara
Raspberry Pi 1	Servidor OPC UA	192.168.10.1	255.255.255.0 (/24)
Raspberry Pi 2	Cliente OPC UA	192.168.10.2	255.255.255.0 (/24)

La configuración puede hacerse con NetworkManager (`nmcli` o la herramienta gráfica de Raspberry Pi OS). Debe documentarse el procedimiento empleado.

3. **Verificación:** verificar el enlace en cada capa y documentar los resultados:
   - estado del enlace, velocidad y modo dúplex negociados (`ethtool eth0` o `ip link`);
   - direcciones asignadas (`ip addr`);
   - conectividad (`ping`);
   - tabla ARP luego del ping (`ip neigh`).
4. **Convivencia con Wi-Fi:** las placas pueden mantener el Wi-Fi conectado para tener acceso a internet y por SSH. Todo el tráfico OPC UA entre ambas debe circular por `eth0`, y el informe debe demostrarlo.
5. ***Pregunta de análisis:*** explicar teóricamente qué se debe realizar para montar una red Ethernet punto a punto entre dos equipos sin dispositivos intermedios. Desarrollar:
   - en la capa física: tipo de cable, función de auto MDI-X y autonegociación de velocidad y dúplex;
   - en la capa de enlace: direcciones MAC y el rol de ARP;
   - en la capa de red: direccionamiento estático, máscara de subred, por qué no es necesaria una puerta de enlace y qué ocurriría sin IP fija (direcciones de enlace local 169.254.0.0/16);
   - qué herramienta permite verificar cada capa.

PARTE 2: Servidor OPC UA y modelo de información

1. **Desarrollo del servidor (Raspberry Pi 1):** implementar un servidor OPC UA con python-asyncua, open62541 o equivalente. El servidor debe exponer el endpoint `opc.tcp://192.168.10.1:4840` y escuchar en todas las interfaces.
2. **Modelo de información con NodeIds fijos:** registrar el espacio de nombres `urn:unraf:pdci:t2` y crear, bajo la carpeta *Objects*, dos objetos con las siguientes variables:

Objeto	Variable	NodeId	Tipo de dato	Acceso	Origen del dato
Servidor	EstadoPulsador	ns=1;s=Servidor.EstadoPulsador	Boolean	Lectura	Pulsador de la Raspberry Pi 1
Servidor	Rampa	ns=1;s=Servidor.Rampa	UInt16	Lectura	Generada por la Raspberry Pi 1
Cliente	EstadoPulsador	ns=1;s=Cliente.EstadoPulsador	Boolean	Lectura/ escritura	Escrita por la Raspberry Pi 2
Cliente	Rampa	ns=1;s=Cliente.Rampa	UInt16	Lectura/ escritura	Escrita por la Raspberry Pi 2

Agregar a ambas variables `Rampa` la propiedad `EURange` (0 a 65535).

3. **Rampa del servidor:** generar un **diente de sierra ascendente** de 0 a 65535 con período de 4 segundos, actualizado cada 50 ms. Al llegar al máximo, el valor vuelve a 0.
4. **Salidas del servidor:** manejar los LEDs de la Raspberry Pi 1 con los datos que escribe el cliente:
    - el LED digital refleja `Cliente.EstadoPulsador`;
    - el LED PWM varía su brillo según `Cliente.Rampa`, escalada a 0–255.
5. **Verificación con cliente genérico:** desde la PC, conectarse al servidor con UaExpert en modo de seguridad *None*. La PC puede alcanzar el servidor por la IP Wi-Fi de la Raspberry Pi 1.

- Recorrer el espacio de direcciones (*Browse*) y documentar los atributos de cada variable: NodeId, BrowseName, DataType, AccessLevel, Value, StatusCode, SourceTimestamp y ServerTimestamp.
- Visualizar `Servidor.Rampa` en la vista de tendencia de UaExpert y verificar su forma de onda.
- Escribir `Cliente.EstadoPulsador` y `Cliente.Rampa` desde UaExpert y verificar la respuesta de los LEDs.
- Intentar escribir una variable de solo lectura y registrar el StatusCode devuelto.

6. ***Pregunta de análisis:*** ¿qué diferencia conceptual existe entre el par cliente-servidor de OPC UA y el par maestro-esclavo de MODBus? ¿Qué ventaja aporta navegar el espacio de direcciones frente a conocer de antemano una tabla de registros?

PARTE 3: Cliente OPC UA e intercambio bidireccional

1. **Desarrollo del cliente (Raspberry Pi 2):** programar un cliente OPC UA que se conecte al endpoint del servidor a través de `eth0`.
2. **Rampa del cliente:** generar una **rampa triangular** de 0 a 65535 con período de 8 segundos (4 s de subida y 4 s de bajada), actualizada cada 50 ms. Las formas de onda distintas permiten distinguir a simple vista, en cada placa, que el brillo del LED PWM proviene del otro dispositivo y no de su propio generador.
3. **Lógica de intercambio:**
   - **Lectura y replicación:** obtener `Servidor.EstadoPulsador` y `Servidor.Rampa`. Reflejarlos en el LED digital y en el LED PWM de la Raspberry Pi 2, respectivamente.
   - **Escritura:** escribir el estado del pulsador local en `Cliente.EstadoPulsador` (al cambiar) y la rampa local en `Cliente.Rampa` (cada 100 ms).
4. **Lectura cíclica frente a suscripción:** implementar la lectura del servidor de dos formas y capturar ambas durante 60 segundos:
   - **Modo A:** servicio *Read* periódico (*polling*) cada 100 ms.
   - **Modo B:** *CreateSubscription* + *CreateMonitoredItems* con intervalo de publicación de 100 ms. Documentar los parámetros SamplingInterval, PublishingInterval y QueueSize.

- Comparar la cantidad de tramas y de bytes en ambos modos.
- Explicar el mecanismo PublishRequest / PublishResponse, incluyendo los mensajes KeepAlive. Para observarlos, repetir la captura con el pulsador en reposo y la rampa detenida.
- Medir la latencia entre el accionamiento del pulsador en una placa y el encendido del LED en la otra.

5. **Registro en consola:** ambas placas deben imprimir periódicamente el valor de rampa que generan, el valor que reciben y el SourceTimestamp del dato recibido.
6. **Prueba de desconexión:** con el sistema en funcionamiento, desconectar el cable Ethernet.
   - Verificar que el LED PWM de cada placa se congela, mientras su generador local continúa.
   - Registrar el StatusCode informado por el cliente y el comportamiento de la sesión.
   - Al reconectar, documentar cómo se recupera la comunicación.
7. **Opcional:** aplicar una corrección gamma al escalar la rampa a 0–255, por ejemplo brillo = 255 · (valor / 65535)^2,2. Justificar por qué una rampa lineal en PWM no se percibe como lineal.

PARTE 4: Análisis de tramas y seguridad

1. **Captura:** un switch no reenvía hacia la PC el tráfico entre las dos placas, y en el enlace directo la PC no participa. Por eso, la captura debe realizarse en una de las Raspberry Pi, por ejemplo con `sudo tcpdump -i eth0 -w captura.pcapng`, y luego analizarse en la PC con Wireshark usando el filtro `opcua`. Capturar en modo de seguridad *None* una sesión completa, desde la conexión del cliente hasta su desconexión.
2. **Secuencia de mensajes:** elaborar un diagrama de secuencia con cada mensaje intercambiado y quién lo origina:
   - resolución ARP y establecimiento TCP;
   - HEL/ACK y OPN (OpenSecureChannel);
   - GetEndpoints, CreateSession y ActivateSession;
   - CreateSubscription, CreateMonitoredItems, Publish y Write;
   - cierre (CloseSession y CLO).
3. Explicar la función de cada etapa y la diferencia entre *conexión TCP*, *canal seguro* y *sesión*.

4. **Encapsulamiento completo:** para una trama que transporte un PublishResponse con el valor de `Servidor.Rampa`, identificar y explicar cada nivel:
    - cabecera Ethernet II: MAC de destino y origen, EtherType;
    - cabecera IPv4;
    - cabecera TCP, con sus puertos y números de secuencia;
    - mensaje OPC UA.
5. **Decodificación byte a byte:** para un par **WriteRequest / WriteResponse** sobre `Cliente.Rampa` y otro sobre `Cliente.EstadoPulsador`, presentar el volcado hexadecimal y marcar cada campo con su valor, tipo y significado:
    - cabecera de mensaje (MessageType, ChunkType, MessageSize) y SecureChannelId;
    - cabecera de seguridad simétrica (TokenId) y cabecera de secuencia (SequenceNumber, RequestId);
    - NodeId del tipo de servicio y RequestHeader (AuthenticationToken, Timestamp, RequestHandle, TimeoutHint);
    - parámetros del servicio: NodeId de la variable, AttributeId y DataValue con su máscara de codificación;
    - Variant: byte de tipo y valor. Explicar cómo se codifica la word en *little-endian* y cómo un Boolean.
6. **Eficiencia de la trama:** calcular, para la transmisión de una sola word, la relación entre bytes útiles y bytes totales, considerando las cabeceras Ethernet, IP, TCP y OPC UA. Comparar con la trama MODBus RTU equivalente de la Tarea Nº1 y discutir el resultado.
7. **Modos y políticas de seguridad:** configurar el servidor con al menos tres endpoints y repetir la conexión del cliente con cada uno:
    - SecurityMode *None*;
    - SecurityMode *Sign*, política Basic256Sha256;
    - SecurityMode *SignAndEncrypt*, política Basic256Sha256.
8. Para cada modo:
    - explicar el rol de los certificados X.509 de aplicación y el proceso de confianza (*trust list*);
    - mostrar en Wireshark qué campos de la trama siguen siendo legibles y cuáles quedan cifrados;
    - medir si cambia la latencia del intercambio.

9. **Autenticación de usuario:** habilitar autenticación con usuario y contraseña, además del acceso anónimo. Explicar en qué mensaje viaja el *token* de identidad y cómo se protege.

REQUERIMIENTOS ADICIONALES DEL TRABAJO

1. **Diagrama de red:** esquema del enlace punto a punto con interfaces, direcciones IP y conexionado de los periféricos de cada placa.
2. **Diagrama de flujo:** del servidor (inicialización, creación del espacio de direcciones, generación de la rampa, lectura del pulsador, atención de escrituras y manejo de LEDs) y del cliente (conexión, suscripción, manejo de notificaciones, escrituras y reconexión ante pérdida del canal).
3. **Trazabilidad de código:** los fragmentos de código de cada dispositivo deben explicarse referenciando el bloque del diagrama de flujo al que pertenecen.
4. **Capturas:** incluir los archivos `.png` utilizados junto al informe.

Evaluación y defensa

La evaluación de este Trabajo Práctico consta de dos instancias obligatorias y complementarias: la entrega de un Informe Técnico y una Defensa Oral y Demostrativa.

Informe Técnico (Escrito)

El documento debe entregarse en formato PDF y cumplir con los estándares de redacción académica. Se evaluará la capacidad de síntesis, el rigor técnico y la claridad en los siguientes puntos:

- **Arquitectura de red y capa física:** configuración del enlace punto a punto, direccionamiento, verificación por capas y conexionado de cada placa.
- **Modelo de información:** tabla del espacio de direcciones con NodeIds, tipos de datos, niveles de acceso y servicios OPC UA utilizados.
- **Análisis de tramas:** diagrama de secuencia, encapsulamiento, decodificación byte a byte y comparación de modos de seguridad.
- **Diseño lógico y diagramas de flujo** del servidor y del cliente.

Defensa Oral (Presentación de Resultados)

La defensa tendrá una duración estricta de 12 minutos, seguida de una sesión de preguntas por parte de la cátedra. El grupo deberá gestionar el tiempo con precisión, aplicando técnicas de comunicación eficaz con las que todos los miembros deben ser capaces de hablar. La estructura de la exposición debe seguir la siguiente secuencia retórica:

- Apertura y Exordio (2 minutos):
    - Presentación del problema: La necesidad de coherencia de estado en sistemas maestro-esclavo.
    - Establecimiento de los objetivos del diseño.

Se evaluará la capacidad de captar la atención mediante un discurso fluido y la definición clara del alcance del proyecto.

- Desarrollo y Cuerpo Técnico (8 minutos):
    - Explicación de la arquitectura del sistema y flujo de datos.
    - Justificación de librerías utilizadas y desafíos encontrados en la programación (ej. colisiones en el canal IR o latencias).
    - Demostración funcional: Ejecución en tiempo real de la secuencia de colores y la recuperación de estado ante desconexión.

Se observará la precisión terminológica (uso correcto de conceptos como bitrate, modulación, buffer, etc.) y la coordinación del equipo. El apoyo visual (diapositivas o esquemas) debe ser complementario y no una lectura del código.

- Cierre y Peroración (2 minutos):
    - Síntesis de los resultados obtenidos y validación de la hipótesis inicial (sincronización lograda).
    - Propuesta de mejoras o escalabilidad del protocolo diseñado.

El cierre debe retomar los puntos planteados en la apertura, dejando una conclusión sólida que refuerce la resolución exitosa del problema de ingeniería.

Criterios de Evaluación General

Para la calificación final se ponderarán los siguientes aspectos:

- Funcionalidad (35%): El sistema cumple con la sincronización RGB, el protocolo IR es estable y la persistencia de datos es efectiva.
- Informe Técnico (35%): Calidad de la documentación, diagramación lógica y análisis de señales.

Competencia Comunicativa y Defensa (30%): Claridad en la exposición oral, manejo del tiempo (12 min), uso de vocabulario técnico adecuado y capacidad de respuesta ante interrogantes técnicas.

Entrega

El trabajo deberá ser realizado en grupos de hasta 3 personas las cuales deberán trabajar en conjunto y ser capaces de explicar el funcionamiento a la hora de presentar el trabajo.

Se pondrá a disposición foros en la plataforma para consultas y se dispondrá de un espacio dentro de las clases de práctica determinadas para revisar dificultades encontradas durante la programación y/o ejecución del circuito.

La entrega deberá contar con un informe detallado de los conceptos, procedimientos y algoritmos empleados junto a la presentación y defensa del circuito funcional. La justificación teórica de los distintos apartados deberá ser realizada en base a bibliografía. El archivo deberá respetar los siguientes criterios:

- Formato: PDF
- Nombre del archivo: Apellido1_Apellido2_Apellido3_PdCI_T2_26.pdf
- Texto justificado
- Márgenes: Laterales: 3. Superior e inferior 2.5
- Tipografía y tamaño: Arial Tamaño 11
- Interlineado 1.5, espaciado posterior 6 y anterior 0
- Referencias y citas bibliográficas: APA 7ma edición.

Se deberán incluir, junto al informe, los códigos, simulaciones y diagramas que hayan sido empleados en el cumplimiento de la consigna.

Aclaraciones importantes:

- El informe debe demostrar entendimiento de lo que se ha realizado.
    - En el informe se pueden obviar sólo la declaración de variables, las cuales se recomienda comentar en el código adjunto.
    - Es fundamental incluir el conexionado de cada bloque del circuito.
    - En el caso de emplear librerías para la programación, especificar qué parámetros requiere la configuración de las mismas, qué funciones se emplean y qué parámetros (con tipo) requieren, qué y cómo devuelven como resultado.
- El día de la defensa, todos los integrantes deben presentarse con acceso al informe entregado en formato papel o digital y deben ser capaces de responder cuestiones pertinentes a la realización del trabajo y la teoría puesta en juego.
- Tanto la entrega del informe como la defensa del proyecto en funcionamiento serán instancias de evaluación, por lo tanto, se recuerda respetar los formatos establecidos.

Fechas

La entrega del informe preliminar se realiza a través de la plataforma en la fecha estipulada.

- Debe constar de la explicación detallada de cómo piensan realizar la solución de las consignas junto al diagrama de flujo que muestre gráficamente cómo trabajará la solución que se desarrollará.
- Debe describir el tipo de comunicación elegida y el orden de los datos dentro de las tramas enviadas.

La defensa del circuito se desarrollará de forma presencial en la fecha estipulada en el cronograma y en el lugar determinado en clases.

Se coordinará el horario por grupo de los coloquios.